

Manual — Versión Móvil

TFG DAM — Stockly

Adrián Bravo Santos · Miguel Ángel Florido
Generado el 2026-05-16



Guía: versión móvil

El frontend ya es **responsive + PWA**, así que móvil ≠ proyecto aparte. Esta guía resume:

1. Cómo se comporta la app en móvil hoy.
2. Cómo instalarla como PWA en Android e iOS.
3. Cómo empaquetarla como **APK Android** real con **Capacitor** (sin reescribir nada).

1. Comportamiento responsive actual

Punto de quiebre principal: **880 px**.

Pantalla	Comportamiento
> 880 px	Topbar con tabs centradas, grid de productos auto-fill (mín. 240 px)
480-880 px	Bottom navigation, topbar reducida, grid 2 columnas
< 480 px	Stat cards 2×, modal con campos en una columna, modales a ancho completo

La hoja de estilos usa variables CSS y `safe-area-inset-bottom` para respetar la barra de gestos de iOS.

2. Instalar como PWA

Android (Chrome / Edge)

1. Abre `https://tfg.tudominio.com`.
2. Menú : → **Instalar aplicación** (o aparece automáticamente un banner).
3. La app se añade al cajón con su icono y se abre en pantalla completa, sin barra del navegador.

iOS (Safari ≥ 16)

1. Abre la URL en Safari.
2. Botón **Compartir** → **Añadir a pantalla de inicio**.
3. iOS no soporta todas las features PWA (push limitado), pero la app se ve como nativa.

Escritorio

Edge / Chrome / Brave: icono ⊕ en la barra de direcciones → **Instalar Stockly**.

3. Convertir la PWA en APK / IPA con Capacitor

Capacitor envuelve cualquier web app en una WebView nativa. Es lo más rápido para tener una APK que enseñar en la defensa.

3.1 Requisitos

- Node 18+
- **Android Studio** instalado (incluye SDK + JDK).
- (iOS) macOS + Xcode si quieres `.ipa`.

3.2 Setup del proyecto Capacitor

Desde la raíz del repo:

```
mkdir mobile && cd mobile
npm init -y
npm i @capacitor/core @capacitor/cli @capacitor/android
npx cap init "Stockly" "app.stockly" --web-dir ../frontend
```

Edita `mobile/capacitor.config.json`:

```
{
  "appId": "app.stockly",
  "appName": "Stockly",
  "webDir": "../frontend",
  "server": {
    "url": "https://tfg.tudominio.com",
    "cleartext": false
  }
}
```

Con `server.url` apuntando al dominio real, la APK funciona como un navegador kiosko siempre actualizado. Sin esa clave, empaqueta los archivos estáticos y se queda fija a esa versión.

3.3 Generar la APK

```
npx cap add android
npx cap sync android
npx cap open android      # se abre Android Studio
```

En Android Studio:

1. **Build** → **Generate Signed Bundle / APK** → APK.
2. Crea un keystore nuevo y guarda la contraseña en el gestor de contraseñas.

3. La APK queda en `mobile/android/app/release/app-release.apk` — instalable en cualquier Android.

3.4 iOS (opcional)

```
npm i @capacitor/ios
npx cap add ios
npx cap open ios          # abre Xcode
# Product → Archive → Distribute → Development
```

4. Mejoras móvil-first pendientes (ver ROADMAP §5-6)

#	Mejora	Beneficio móvil
1	Lector de códigos QR/barras con <code>BarcodeDetector</code> o <code>@capacitor/barcode-scanner</code>	Buscar producto por SKU físico
2	Push notifications con OneSignal o Firebase	Avisar al cliente cuando su reserva pase a "Confirmada"
3	Pull-to-refresh en la lista de productos y reservas	Patrón nativo móvil
4	Modo offline real (no sólo shell) cacheando últimos productos vistos	Trabajadores en zonas con poca señal
5	Compartir una reserva (Web Share API) → manda el detalle por WhatsApp	Fácil coordinación
6	Geolocalización del trabajador para asignar reservas al más cercano	Almacenes grandes

Cada item se puede convertir en una historia de usuario para Sprints 6-8.

6. App móvil dedicada para el empleado (Fase 8 del roadmap)

La PWA cubre al cliente final. Para el **operario de almacén** queremos una app instalable orientada a su flujo de trabajo: ver lo que tiene asignado, confirmar entregas y reportar incidencias sin pelearse con un navegador.

6.1 Objetivo

Un operario abre la app, hace login una vez (biometría después) y ve:

1. **Reservas asignadas a él**, agrupadas por **ubicación en almacén** (pasillo / estantería) para hacer la ronda óptima.
2. Al entrar en una reserva: detalle del producto, cantidad, cliente, y dos botones grandes:

-  **Confirmar entrega** (opcional: foto del paquete, firma del cliente).
-  **Dar incidencia** (rotura, faltante, mal estado, dirección incorrecta).

6.2 Arquitectura propuesta

```
mobile/                ← proyecto Capacitor independiente
  capacitor.config.json ← server.url = https://tfg.tudominio.com/empleado
  android/             ← proyecto Gradle generado
  ios/                 ← opcional
frontend/empleado/     ← nueva sub-SPA o vista filtrada de la PWA actual
backend/src/routes/
  reservas.js          ← + PATCH /:id/entregar, POST /:id/incidencias
  incidencias.js (nuevo) ← CRUD de incidencias
db/schema.sql
+ tabla incidencias    ← (id, reserva_id, operario_id, tipo, descripcion, fotos JSON, creada_en)
```

La sub-SPA `/empleado` es la misma base de código que el frontend actual pero con:

- Bottom-nav reducida (Reservas / Incidencias / Perfil).
- Sin catálogo de productos público.
- Layout listas grandes táctiles (mínimo 56 px de alto por fila).

Capacitor solo añade el shell nativo + permisos (cámara, push, biometría).

6.3 Endpoints nuevos

Método	Ruta	Body	Resultado
GET	<code>/api/reservas/mias</code>	—	Reservas con <code>operario_id = req.user.id</code> agrupadas por ubicación
PATCH	<code>/api/reservas/:id/entregar</code>	<code>{ foto?: base64, firma?: base64 }</code>	Estado → <code>entregada</code> , registra movimiento + adjuntos
POST	<code>/api/reservas/:id/incidencias</code>	<code>{ tipo, descripcion, fotos?: [base64] }</code>	Crea incidencia, marca reserva como <code>con_incidencia</code>
GET	<code>/api/incidencias?abiertas=true</code>	—	Para el panel admin
PATCH	<code>/api/incidencias/:id/resolver</code>	<code>{ resolucion }</code>	Cierra incidencia

6.4 Esquema de la tabla `incidencias`

```
CREATE TABLE incidencias (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  reserva_id  INT NOT NULL,  
  operario_id INT NOT NULL,  
  tipo        ENUM('rotura','faltante','mal_estado','direccion','otro') NOT NULL,  
  descripcion TEXT,  
  fotos       JSON,  
  estado      ENUM('abierta','resuelta') NOT NULL DEFAULT 'abierta',  
  resolucion  TEXT,  
  creada_en   DATETIME DEFAULT CURRENT_TIMESTAMP,  
  resuelta_en DATETIME NULL,  
  FOREIGN KEY (reserva_id) REFERENCES reservas(id),  
  FOREIGN KEY (operario_id) REFERENCES usuarios(id)  
);
```

6.5 Roadmap de implementación (4 semanas)

Semana	Trabajo
1	Migración SQL <code>incidencias</code> , endpoints REST, tests Jest
2	Sub-SPA <code>/empleado</code> (lista, detalle, confirmación, formulario incidencia)
3	Bootstrap Capacitor + permisos cámara + APK firmada
4	Login biométrico, push (FCM), modo offline básico (cola local)

6.6 Mejoras opcionales

- Geolocalización para detectar que el operario está realmente en el almacén al confirmar.
- Escáner QR para validar que el producto entregado coincide con el reservado.
- Modo "ruta optimizada": ordena las reservas por proximidad de ubicaciones (algoritmo TSP simple sobre la lista).

5. Mini checklist para la defensa con móvil

- ☐ Móvil con la PWA instalada y datos demo cargados.
- ☐ APK generada y compartida por AirDrop / cable USB en otro dispositivo.
- ☐ Wifi del centro testado para que la API responda durante la defensa.
- ☐ Modo avión activado para mostrar que la PWA al menos abre la shell offline.
- ☐ Capturas en `docs/diagramas/mobile/` para incluir en la memoria.

